

Start coding or [generate](#) with AI.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
sales = pd.read_csv("/kaggle/bm_sales.csv")
skus = pd.read_csv("/kaggle/bm_skus.csv")

sales["date"] = pd.to_datetime(sales["date"])

print("Sales:", sales.shape)
print("SKUs:", skus.shape)
```

```
Sales: (641843, 9)
SKUs: (200, 7)
```

```
sales_sku = sales.merge(
    skus,
    on="sku_id",
    how="left"
)

print("Merged dataset shape:", sales_sku.shape)
```

```
Merged dataset shape: (641843, 15)
```

```
print("Sales records:", len(sales))
print("Merged records:", len(sales_sku))

print("Missing SKU names:", sales_sku["sku_name"].isna().sum())
print("Missing categories:", sales_sku["category"].isna().sum())
```

```
Sales records: 641843
Merged records: 641843
Missing SKU names: 0
Missing categories: 0
```

daily sales-> weekly sku demand

```
# Create week column
```

```
sales_sku["week"] = (  
    sales_sku["date"]  
    .dt.to_period("W-SUN")  
    .dt.start_time  
)
```

```
print(sales_sku[["date", "week", "sku_id", "quantity"]].head())
```

|   | date       | week       | sku_id | quantity |
|---|------------|------------|--------|----------|
| 0 | 2021-01-01 | 2020-12-28 | 1124   | 1        |
| 1 | 2021-01-01 | 2020-12-28 | 1035   | 3        |
| 2 | 2021-01-01 | 2020-12-28 | 1088   | 1        |
| 3 | 2021-01-01 | 2020-12-28 | 1164   | 6        |
| 4 | 2021-01-01 | 2020-12-28 | 1093   | 1        |

```
# Aggregate sales to weekly SKU level
```

```
weekly_sales = (  
    sales_sku  
    .groupby(["week", "sku_id"], as_index=False)  
    .agg(  
        demand=("quantity", "sum"),  
        sales_value=("total_value", "sum"),  
        avg_price=("unit_price_x", "mean"),  
        avg_discount=("discount_pct", "mean")  
    )  
)
```

```
print("Weekly SKU dataset shape:", weekly_sales.shape)  
print(weekly_sales.head())
```

```
Weekly SKU dataset shape: (50600, 6)
```

|   | week       | sku_id | demand | sales_value | avg_price  | avg_discount |
|---|------------|--------|--------|-------------|------------|--------------|
| 0 | 2020-12-28 | 1001   | 39     | 5751.48     | 146.931429 | 10.714286    |
| 1 | 2020-12-28 | 1002   | 35     | 435.23      | 12.585556  | 8.333333     |
| 2 | 2020-12-28 | 1003   | 42     | 469.80      | 11.169231  | 6.923077     |
| 3 | 2020-12-28 | 1004   | 15     | 342.54      | 22.960000  | 7.500000     |
| 4 | 2020-12-28 | 1005   | 17     | 282.28      | 16.560000  | 6.428571     |

```
print("Number of weeks:", weekly_sales["week"].nunique())  
print("Number of SKUs:", weekly_sales["sku_id"].nunique())
```

```
print("\nDate range:")  
print("Start:", weekly_sales["week"].min())  
print("End:", weekly_sales["week"].max())
```

```
print("\nMissing values:")  
print(weekly_sales.isnull().sum())
```

```
Number of weeks: 253  
Number of SKUs: 200
```

```
Date range:
```

```
Start: 2020-12-28 00:00:00
End: 2025-10-27 00:00:00
```

Missing values:

```
week          0
sku_id        0
demand        0
sales_value   0
avg_price     0
avg_discount  0
dtype: int64
```

hamro columns name kk x vana r check gara ko ho unit\_price\_x and y

```
print(sales_sku.columns.tolist())
```

```
['date', 'store_id', 'sku_id', 'customer_id', 'quantity', 'unit_price_x', 'to
```

Start coding or [generate](#) with AI.

complete SKU \* week grid

```
# Create complete SKU x Week combinations

all_weeks = pd.date_range(
    start=weekly_sales["week"].min(),
    end=weekly_sales["week"].max(),
    freq="7D"
)

all_skus = weekly_sales["sku_id"].unique()

complete_grid = pd.MultiIndex.from_product(
    [all_weeks, all_skus],
    names=["week", "sku_id"]
).to_frame(index=False)

print("Complete grid shape:", complete_grid.shape)
print("Total weeks:", len(all_weeks))
print("Total SKUs:", len(all_skus))
```

```
Complete grid shape: (50600, 2)
Total weeks: 253
Total SKUs: 200
```

```
weekly_sales_complete = complete_grid.merge(
    weekly_sales,
    on=["week", "sku_id"],
    how="left"
)

print("Complete weekly dataset shape:",
```

```
weekly_sales_complete.shape)

print("\nMissing values:")
print(weekly_sales_complete.isnull().sum())
```

Complete weekly dataset shape: (50600, 6)

```
Missing values:
week          0
sku_id        0
demand        0
sales_value    0
avg_price     0
avg_discount   0
dtype: int64
```

```
# Fill zero-sales weeks

zero_columns = [
    "demand",
    "sales_value",
    "avg_price",
    "avg_discount"
]

weekly_sales_complete[zero_columns] = (
    weekly_sales_complete[zero_columns].fillna(0)
)

print("Zero-sales weeks handled successfully.")

print("\nMissing values after filling:")
print(weekly_sales_complete.isnull().sum())
```

Zero-sales weeks handled successfully.

```
Missing values after filling:
week          0
sku_id        0
demand        0
sales_value    0
avg_price     0
avg_discount   0
dtype: int64
```

```
weekly_sales_complete = weekly_sales_complete.merge(
    skus[
        [
            "sku_id",
            "sku_name",
            "category",
            "subcategory",
            "brand",
            "cost_price"
        ]
    ]
```

```

],
on="sku_id",
how="left"
)

print("Final weekly dataset shape:",
      weekly_sales_complete.shape)

print(weekly_sales_complete.head())

```

```

Final weekly dataset shape: (50600, 11)
   week  sku_id  demand  sales_value  avg_price  avg_discount  \
0 2020-12-28   1001     39      5751.48  146.931429    10.714286
1 2020-12-28   1002     35       435.23   12.585556     8.333333
2 2020-12-28   1003     42       469.80   11.169231     6.923077
3 2020-12-28   1004     15       342.54   22.960000     7.500000
4 2020-12-28   1005     17       282.28   16.560000     6.428571

   sku_name  category  subcategory  \
0  Electronics_Chargers_1001  Electronics  Chargers
1  Dairy_Cream_1002  Dairy  Cream
2  Household_Cleaning_Supplies_1003  Household  Cleaning Supplies
3  Beverages_Juices_1004  Beverages  Juices
4  Household_Paper_Products_1005  Household  Paper Products

   brand  cost_price
0  Budget    103.44
1  Local     8.96
2  BlueMart  7.47
3  International  17.58
4  Budget    13.68

```

Start coding or [generate](#) with AI.

## BASIC EDA

```

print("Dataset shape:", weekly_sales_complete.shape)

print("\nColumns:")
print(weekly_sales_complete.columns.tolist())

print("\nData types:")
print(weekly_sales_complete.dtypes)

```

```

Dataset shape: (50600, 11)

Columns:
['week', 'sku_id', 'demand', 'sales_value', 'avg_price', 'avg_discount', 'sku

Data types:
week          datetime64[ns]
sku_id        int64
demand        int64
sales_value    float64
avg_price      float64
avg_discount   float64

```

```
sku_name      object
category      object
subcategory    object
brand         object
cost_price    float64
dtype: object
```

```
print("Demand Statistics")
print("=" * 50)

print(weekly_sales_complete["demand"].describe())
```

```
Demand Statistics
=====
count      50600.000000
mean        33.146502
std         16.930454
min          1.000000
25%         20.000000
50%         30.000000
75%         43.000000
max        131.000000
Name: demand, dtype: float64
```

Start coding or [generate](#) with AI.

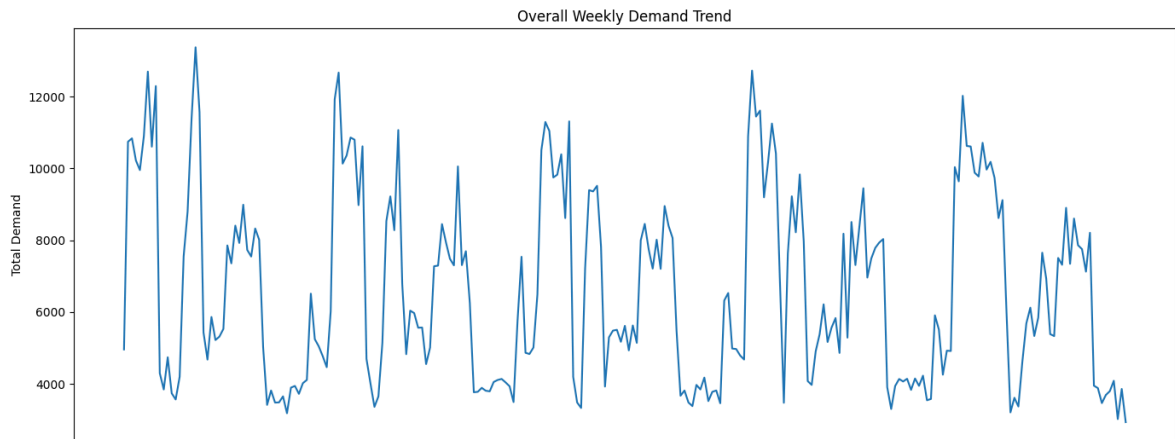
overall weekly demand

```
weekly_demand = (
    weekly_sales_complete
    .groupby("week", as_index=False)["demand"]
    .sum()
)

plt.figure(figsize=(14, 6))

plt.plot(
    weekly_demand["week"],
    weekly_demand["demand"]
)

plt.title("Overall Weekly Demand Trend")
plt.xlabel("Week")
plt.ylabel("Total Demand")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



```
top_skus = (
    weekly_sales_complete
    .groupby(["sku_id", "sku_name"], as_index=False)["demand"]
    .sum()
    .sort_values("demand", ascending=False)
    .head(10)
)

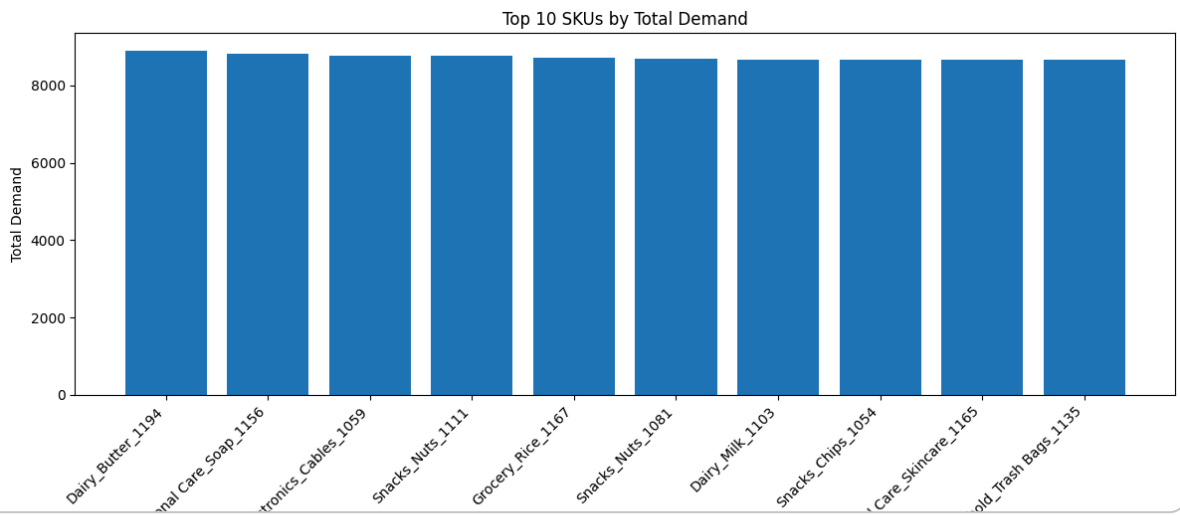
print(top_skus)
```

|     | sku_id | sku_name                    | demand |
|-----|--------|-----------------------------|--------|
| 193 | 1194   | Dairy_Butter_1194           | 8904   |
| 155 | 1156   | Personal_Care_Soap_1156     | 8821   |
| 58  | 1059   | Electronics_Cables_1059     | 8767   |
| 110 | 1111   | Snacks_Nuts_1111            | 8754   |
| 166 | 1167   | Grocery_Rice_1167           | 8724   |
| 80  | 1081   | Snacks_Nuts_1081            | 8701   |
| 102 | 1103   | Dairy_Milk_1103             | 8676   |
| 53  | 1054   | Snacks_Chips_1054           | 8673   |
| 164 | 1165   | Personal_Care_Skincare_1165 | 8660   |
| 134 | 1135   | Household_Trash_Bags_1135   | 8654   |

```
plt.figure(figsize=(12, 6))

plt.bar(
    top_skus["sku_name"],
    top_skus["demand"]
)

plt.title("Top 10 SKUs by Total Demand")
plt.xlabel("SKU")
plt.ylabel("Total Demand")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```



Start coding or [generate](#) with AI.

## Category-wise Demand

```
category_demand = (
    weekly_sales_complete
    .groupby("category", as_index=False)["demand"]
    .sum()
    .sort_values("demand", ascending=False)
)

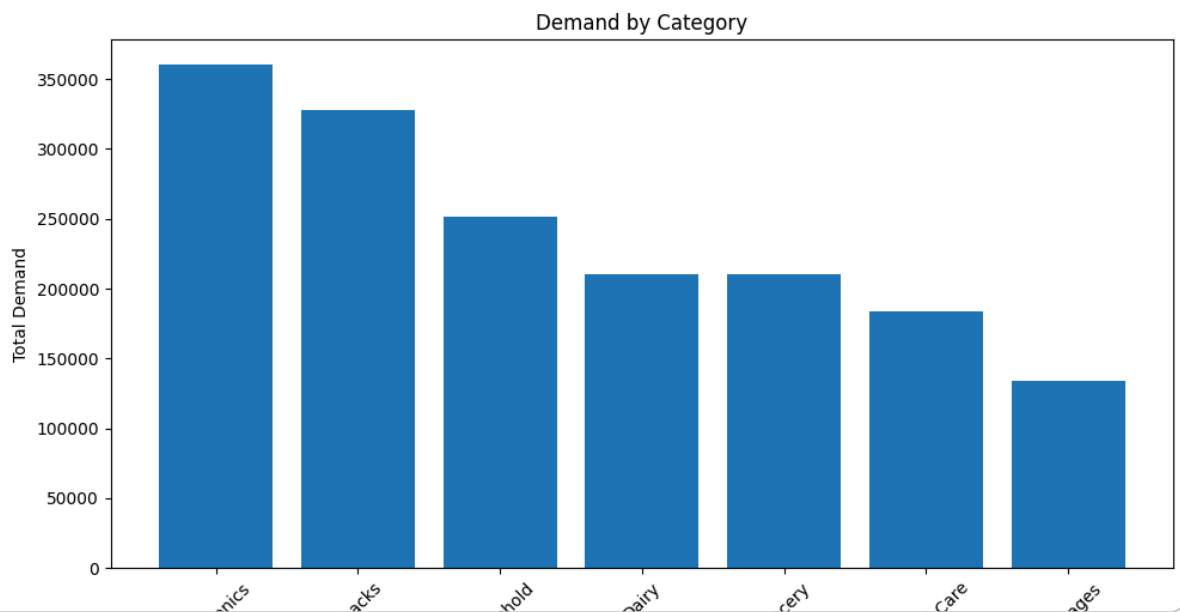
print(category_demand)
```

|   | category      | demand |
|---|---------------|--------|
| 2 | Electronics   | 360176 |
| 6 | Snacks        | 327472 |
| 4 | Household     | 251917 |
| 1 | Dairy         | 210199 |
| 3 | Grocery       | 209937 |
| 5 | Personal Care | 183865 |
| 0 | Beverages     | 133647 |

```
plt.figure(figsize=(10, 6))

plt.bar(
    category_demand["category"],
    category_demand["demand"]
)

plt.title("Demand by Category")
plt.xlabel("Category")
plt.ylabel("Total Demand")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Start coding or [generate](#) with AI.

## Slow-moving / Dead Stock

```
sku_demand = (
    weekly_sales_complete
    .groupby(["sku_id", "sku_name", "category"], as_index=False)
    .agg(
        total_demand=("demand", "sum"),
        avg_weekly_demand=("demand", "mean")
    )
)

print(sku_demand.sort_values("total_demand").head(10))
```

|     | sku_id | sku_name                      | category      | total_demand | \ |
|-----|--------|-------------------------------|---------------|--------------|---|
| 22  | 1023   | Personal Care_Toothpaste_1023 | Personal Care | 7866         |   |
| 42  | 1043   | Electronics_Batteries_1043    | Electronics   | 7913         |   |
| 75  | 1076   | Snacks_Chocolates_1076        | Snacks        | 8034         |   |
| 18  | 1019   | Electronics_Cables_1019       | Electronics   | 8045         |   |
| 180 | 1181   | Personal Care_Soap_1181       | Personal Care | 8085         |   |
| 90  | 1091   | Dairy_Butter_1091             | Dairy         | 8100         |   |
| 85  | 1086   | Grocery_Rice_1086             | Grocery       | 8113         |   |
| 29  | 1030   | Dairy_Cream_1030              | Dairy         | 8124         |   |
| 161 | 1162   | Dairy_Yogurt_1162             | Dairy         | 8133         |   |
| 57  | 1058   | Personal Care_Soap_1058       | Personal Care | 8133         |   |

|     | avg_weekly_demand |
|-----|-------------------|
| 22  | 31.090909         |
| 42  | 31.276680         |
| 75  | 31.754941         |
| 18  | 31.798419         |
| 180 | 31.956522         |
| 90  | 32.015810         |
| 85  | 32.067194         |
| 29  | 32.110672         |
| 161 | 32.146245         |
| 57  | 32.146245         |

```
# Weekly seasonality analysis

weekly_sales_complete["week_number"] = (
    weekly_sales_complete["week"].dt.isocalendar().week.astype(int)
)

seasonality = (
    weekly_sales_complete
    .groupby("week_number", as_index=False)["demand"]
    .mean()
)

print(seasonality.head(10))
print("\nHighest demand weeks:")
print(
    seasonality
    .sort_values("demand", ascending=False)
    .head(10)
)
```

|   | week_number | demand |
|---|-------------|--------|
| 0 | 1           | 54.088 |
| 1 | 2           | 57.148 |
| 2 | 3           | 54.849 |
| 3 | 4           | 52.286 |
| 4 | 5           | 51.383 |
| 5 | 6           | 53.905 |
| 6 | 7           | 49.213 |
| 7 | 8           | 55.331 |
| 8 | 9           | 30.036 |
| 9 | 10          | 24.991 |

Highest demand weeks:

|    | week_number | demand |
|----|-------------|--------|
| 1  | 2           | 57.148 |
| 7  | 8           | 55.331 |
| 2  | 3           | 54.849 |
| 0  | 1           | 54.088 |
| 5  | 6           | 53.905 |
| 3  | 4           | 52.286 |
| 4  | 5           | 51.383 |
| 6  | 7           | 49.213 |
| 31 | 32          | 41.900 |
| 29 | 30          | 41.267 |

```
print("Lowest demand weeks:")
print(
    seasonality
    .sort_values("demand", ascending=True)
    .head(10)
)
```

Lowest demand weeks:

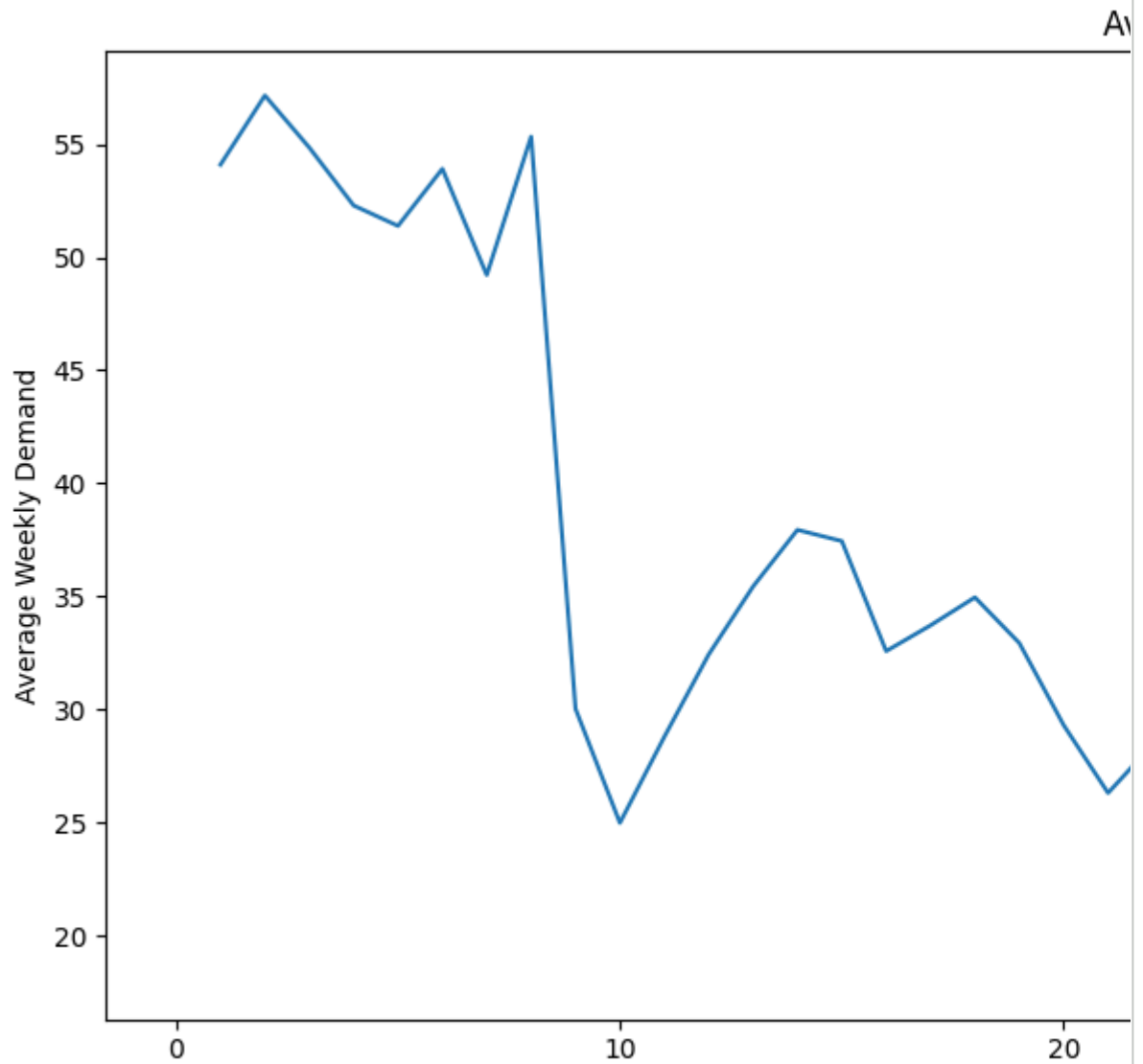
|  | week_number | demand |
|--|-------------|--------|
|--|-------------|--------|

|    |    |          |
|----|----|----------|
| 37 | 38 | 18.25300 |
| 45 | 46 | 18.25875 |
| 43 | 44 | 18.41900 |
| 38 | 39 | 18.50000 |
| 36 | 37 | 18.59100 |
| 35 | 36 | 18.70900 |
| 41 | 42 | 19.03100 |
| 39 | 40 | 19.27700 |
| 40 | 41 | 19.30700 |
| 42 | 43 | 19.60700 |

```
plt.figure(figsize=(14, 6))

plt.plot(
    seasonality["week_number"],
    seasonality["demand"]
)

plt.title("Average Demand by Week of Year")
plt.xlabel("Week Number")
plt.ylabel("Average Weekly Demand")
plt.tight_layout()
plt.show()
```



Start coding or [generate](#) with AI.

```
print("EDA completed successfully!")  
print("Next step: Seasonal Naive Baseline")
```

```
EDA completed successfully!  
Next step: Seasonal Naive Baseline
```

Start coding or [generate](#) with AI.

## Seasonal Naive

```
# Define 6-week test period  
  
test_weeks = (  
    weekly_sales_complete["week"]  
    .sort_values()  
    .unique()[-6:]
```

```
)
```

```
print("Test weeks:")  
print(test_weeks)
```

```
Test weeks:  
<DatetimeArray>  
['2025-09-22 00:00:00', '2025-09-29 00:00:00', '2025-10-06 00:00:00',  
 '2025-10-13 00:00:00', '2025-10-20 00:00:00', '2025-10-27 00:00:00']  
Length: 6, dtype: datetime64[ns]
```

## Seasonal naive forecast

```
# Create seasonal naive forecast using demand from 52 weeks earlier  
  
baseline = weekly_sales_complete[  
    ["week", "sku_id", "demand"]  
].copy()  
  
baseline["forecast"] = baseline.groupby("sku_id")["demand"].shift(52)  
  
baseline_test = baseline[  
    baseline["week"].isin(test_weeks)  
].copy()  
  
print("Baseline test shape:", baseline_test.shape)  
print(baseline_test.head(10))
```

```
Baseline test shape: (1200, 4)
```

|       | week       | sku_id | demand | forecast |
|-------|------------|--------|--------|----------|
| 49400 | 2025-09-22 | 1001   | 12     | 21.0     |
| 49401 | 2025-09-22 | 1002   | 17     | 20.0     |
| 49402 | 2025-09-22 | 1003   | 12     | 18.0     |
| 49403 | 2025-09-22 | 1004   | 31     | 19.0     |
| 49404 | 2025-09-22 | 1005   | 28     | 21.0     |
| 49405 | 2025-09-22 | 1006   | 18     | 20.0     |
| 49406 | 2025-09-22 | 1007   | 26     | 30.0     |
| 49407 | 2025-09-22 | 1008   | 9      | 37.0     |
| 49408 | 2025-09-22 | 1009   | 12     | 18.0     |
| 49409 | 2025-09-22 | 1010   | 30     | 13.0     |

```
print("Missing baseline forecasts:",  
      baseline_test["forecast"].isnull().sum())  
  
print("Actual test records:",  
      baseline_test["demand"].notnull().sum())
```

```
Missing baseline forecasts: 0  
Actual test records: 1200
```

## WAPE Calculate

```

actual = baseline_test["demand"]
forecast = baseline_test["forecast"]

wape = (
    np.sum(np.abs(actual - forecast))
    / np.sum(actual)
) * 100

print(f"Seasonal Naive WAPE: {wape:.2f}%")

```

Seasonal Naive WAPE: 45.56%

```

bias = (
    np.sum(forecast - actual)
    / np.sum(actual)
) * 100

print(f"Seasonal Naive Bias: {bias:.2f}%")

```

Seasonal Naive Bias: 13.50%

```

print("=" * 50)
print("SEASONAL NAIVE BASELINE RESULTS")
print("=" * 50)

print("Forecast horizon: 6 weeks")
print("Seasonal period: 52 weeks")
print(f"WAPE: {wape:.2f}%")
print(f"Bias: {bias:.2f}%")

```

```

=====
SEASONAL NAIVE BASELINE RESULTS
=====
Forecast horizon: 6 weeks
Seasonal period: 52 weeks
WAPE: 45.56%
Bias: 13.50%

```

## feature engineering

```

# Sort data correctly before creating lag features

model_data = weekly_sales_complete.copy()

model_data = model_data.sort_values(
    ["sku_id", "week"]
).reset_index(drop=True)

print("Model data shape:", model_data.shape)

```

Model data shape: (50600, 12)

```
# Create lag features for each SKU

lag_periods = [1, 2, 4, 8, 13, 26, 52]

for lag in lag_periods:
    model_data[f"lag_{lag}"] = (
        model_data.groupby("sku_id")["demand"]
        .shift(lag)
    )

print("Lag features created successfully!")

print(model_data[
    [
        "week",
        "sku_id",
        "demand",
        "lag_1",
        "lag_2",
        "lag_4",
        "lag_8",
        "lag_13",
        "lag_26",
        "lag_52"
    ]
].head(60))
```

|    |      |
|----|------|
| 33 | NaN  |
| 34 | NaN  |
| 35 | NaN  |
| 36 | NaN  |
| 37 | NaN  |
| 38 | NaN  |
| 39 | NaN  |
| 40 | NaN  |
| 41 | NaN  |
| 42 | NaN  |
| 43 | NaN  |
| 44 | NaN  |
| 45 | NaN  |
| 46 | NaN  |
| 47 | NaN  |
| 48 | NaN  |
| 49 | NaN  |
| 50 | NaN  |
| 51 | NaN  |
| 52 | 39.0 |
| 53 | 40.0 |
| 54 | 52.0 |
| 55 | 45.0 |
| 56 | 64.0 |
| 57 | 35.0 |
| 58 | 88.0 |
| 59 | 47.0 |

```
print("\nMissing values in lag features:")

for lag in lag_periods:
    print(
        f"lag_{lag}:",
        model_data[f"lag_{lag}"].isnull().sum()
    )
```

```
Missing values in lag features:
lag_1: 200
lag_2: 400
lag_4: 800
lag_8: 1600
lag_13: 2600
lag_26: 5200
lag_52: 10400
```

## Rolling features

```
# Create rolling demand features

rolling_windows = [4, 8, 13]

for window in rolling_windows:
    model_data[f"rolling_mean_{window}"] = (
        model_data.groupby("sku_id")["demand"]
        .transform(
```

```

        lambda x: x.shift(1).rolling(window).mean()
    )
)

print("Rolling features created successfully!")

print(
    model_data[
        [
            "week",
            "sku_id",
            "demand",
            "rolling_mean_4",
            "rolling_mean_8",
            "rolling_mean_13"
        ]
    ].head(20)
)

```

Rolling features created successfully!

|    | week       | sku_id | demand | rolling_mean_4 | rolling_mean_8 | rolling_mean_13 |
|----|------------|--------|--------|----------------|----------------|-----------------|
| 0  | 2020-12-28 | 1001   | 39     | NaN            | NaN            | Na              |
| 1  | 2021-01-04 | 1001   | 40     | NaN            | NaN            | Na              |
| 2  | 2021-01-11 | 1001   | 52     | NaN            | NaN            | Na              |
| 3  | 2021-01-18 | 1001   | 45     | NaN            | NaN            | Na              |
| 4  | 2021-01-25 | 1001   | 64     | 44.00          | NaN            | Na              |
| 5  | 2021-02-01 | 1001   | 35     | 50.25          | NaN            | Na              |
| 6  | 2021-02-08 | 1001   | 88     | 49.00          | NaN            | Na              |
| 7  | 2021-02-15 | 1001   | 47     | 58.00          | NaN            | Na              |
| 8  | 2021-02-22 | 1001   | 61     | 58.50          | 51.250         | Na              |
| 9  | 2021-03-01 | 1001   | 19     | 57.75          | 54.000         | Na              |
| 10 | 2021-03-08 | 1001   | 20     | 53.75          | 51.375         | Na              |
| 11 | 2021-03-15 | 1001   | 30     | 36.75          | 47.375         | Na              |
| 12 | 2021-03-22 | 1001   | 24     | 32.50          | 45.500         | Na              |
| 13 | 2021-03-29 | 1001   | 13     | 23.25          | 40.500         | 43.38461        |
| 14 | 2021-04-05 | 1001   | 36     | 21.75          | 37.750         | 41.38461        |
| 15 | 2021-04-12 | 1001   | 47     | 25.75          | 31.250         | 41.07692        |
| 16 | 2021-04-19 | 1001   | 46     | 30.00          | 31.250         | 40.69230        |
| 17 | 2021-04-26 | 1001   | 37     | 35.50          | 29.375         | 40.76923        |
| 18 | 2021-05-03 | 1001   | 87     | 41.50          | 31.625         | 38.69230        |
| 19 | 2021-05-10 | 1001   | 30     | 54.25          | 40.000         | 42.69230        |

```
print("Missing values in rolling features:")
```

```

for window in rolling_windows:
    print(
        f"rolling_mean_{window}:",
        model_data[f"rolling_mean_{window}"].isnull().sum()
    )

```

Missing values in rolling features:

```

rolling_mean_4: 800
rolling_mean_8: 1600
rolling_mean_13: 2600

```

## calender feature

```
# Create calendar features

model_data["month"] = model_data["week"].dt.month
model_data["quarter"] = model_data["week"].dt.quarter
model_data["year"] = model_data["week"].dt.year

print("Calendar features created successfully!")

print(
    model_data[
        ["week", "month", "quarter", "year"]
    ].head(15)
)
```

Calendar features created successfully!

|    | week       | month | quarter | year |
|----|------------|-------|---------|------|
| 0  | 2020-12-28 | 12    | 4       | 2020 |
| 1  | 2021-01-04 | 1     | 1       | 2021 |
| 2  | 2021-01-11 | 1     | 1       | 2021 |
| 3  | 2021-01-18 | 1     | 1       | 2021 |
| 4  | 2021-01-25 | 1     | 1       | 2021 |
| 5  | 2021-02-01 | 2     | 1       | 2021 |
| 6  | 2021-02-08 | 2     | 1       | 2021 |
| 7  | 2021-02-15 | 2     | 1       | 2021 |
| 8  | 2021-02-22 | 2     | 1       | 2021 |
| 9  | 2021-03-01 | 3     | 1       | 2021 |
| 10 | 2021-03-08 | 3     | 1       | 2021 |
| 11 | 2021-03-15 | 3     | 1       | 2021 |
| 12 | 2021-03-22 | 3     | 1       | 2021 |
| 13 | 2021-03-29 | 3     | 1       | 2021 |
| 14 | 2021-04-05 | 4     | 2       | 2021 |

Start coding or [generate](#) with AI.

```
import pandas as pd

promotions = pd.read_csv("/kaggle/bm_promotions.csv")

print("Promotions loaded successfully!")
print("Shape:", promotions.shape)
print(promotions.head())
```

Promotions loaded successfully!

Shape: (33, 6)

|   | promo_name                   | start_date | end_date   | discount_pct | \ |
|---|------------------------------|------------|------------|--------------|---|
| 0 | Ramadan Sale 2021            | 2021-04-13 | 2021-05-12 | 25           |   |
| 1 | Eid Al Fitr Sale 2021        | 2021-05-13 | 2021-05-15 | 20           |   |
| 2 | Eid Al Adha Sale 2021        | 2021-07-20 | 2021-07-22 | 25           |   |
| 3 | Dubai Shopping Festival 2021 | 2021-01-01 | 2021-02-28 | 15           |   |
| 4 | Black Friday 2021            | 2021-11-26 | 2021-11-28 | 30           |   |

|   | promo_type | promo_id |
|---|------------|----------|
| 0 | Ramadan    | 1        |

|   |              |   |
|---|--------------|---|
| 1 | Eid          | 2 |
| 2 | Eid          | 3 |
| 3 | DSF          | 4 |
| 4 | Black Friday | 5 |

```
promotions["start_date"] = pd.to_datetime(promotions["start_date"])
promotions["end_date"] = pd.to_datetime(promotions["end_date"])

print("Promotion dates converted successfully!")
print(promotions.dtypes)
```

```
Promotion dates converted successfully!
promo_name      object
start_date      datetime64[ns]
end_date        datetime64[ns]
discount_pct     int64
promo_type      object
promo_id        int64
dtype: object
```

```
print("Number of promotions:", len(promotions))

print("\nPromotion date range:")
print("Start:", promotions["start_date"].min())
print("End:", promotions["end_date"].max())

print("\nPromotion types:")
print(promotions["promo_type"].value_counts())

print("\nPromotions:")
print(promotions)
```

```
Number of promotions: 33
```

```
Promotion date range:
Start: 2021-01-01 00:00:00
End: 2025-08-31 00:00:00
```

```
Promotion types:
promo_type
Eid          10
Ramadan      5
DSF          5
Summer Sale  5
Black Friday  4
National Day  4
Name: count, dtype: int64
```

```
Promotions:
           promo_name start_date  end_date  discount_pct \
0      Ramadan Sale 2021 2021-04-13 2021-05-12         25
1    Eid Al Fitr Sale 2021 2021-05-13 2021-05-15         20
2    Eid Al Adha Sale 2021 2021-07-20 2021-07-22         25
3  Dubai Shopping Festival 2021 2021-01-01 2021-02-28         15
4      Black Friday 2021 2021-11-26 2021-11-28         30
5    UAE National Day 2021 2021-12-01 2021-12-03         20
6      Summer Sale 2021 2021-07-01 2021-08-31         10
```

|    |                         |      |            |            |    |
|----|-------------------------|------|------------|------------|----|
| 7  | Ramadan Sale            | 2022 | 2022-04-02 | 2022-05-01 | 25 |
| 8  | Eid Al Fitr Sale        | 2022 | 2022-05-02 | 2022-05-04 | 25 |
| 9  | Eid Al Adha Sale        | 2022 | 2022-07-10 | 2022-07-12 | 25 |
| 10 | Dubai Shopping Festival | 2022 | 2022-01-01 | 2022-02-28 | 25 |
| 11 | Black Friday            | 2022 | 2022-11-25 | 2022-11-27 | 35 |
| 12 | UAE National Day        | 2022 | 2022-12-01 | 2022-12-03 | 10 |
| 13 | Summer Sale             | 2022 | 2022-07-01 | 2022-08-31 | 10 |
| 14 | Ramadan Sale            | 2023 | 2023-03-23 | 2023-04-20 | 20 |
| 15 | Eid Al Fitr Sale        | 2023 | 2023-04-21 | 2023-04-23 | 25 |
| 16 | Eid Al Adha Sale        | 2023 | 2023-06-29 | 2023-07-01 | 20 |
| 17 | Dubai Shopping Festival | 2023 | 2023-01-01 | 2023-02-28 | 15 |
| 18 | Black Friday            | 2023 | 2023-11-24 | 2023-11-26 | 35 |
| 19 | UAE National Day        | 2023 | 2023-12-01 | 2023-12-03 | 15 |
| 20 | Summer Sale             | 2023 | 2023-07-01 | 2023-08-31 | 20 |
| 21 | Ramadan Sale            | 2024 | 2024-03-11 | 2024-04-09 | 15 |
| 22 | Eid Al Fitr Sale        | 2024 | 2024-04-10 | 2024-04-12 | 20 |
| 23 | Eid Al Adha Sale        | 2024 | 2024-06-17 | 2024-06-19 | 30 |
| 24 | Dubai Shopping Festival | 2024 | 2024-01-01 | 2024-02-28 | 15 |
| 25 | Black Friday            | 2024 | 2024-11-29 | 2024-12-01 | 35 |
| 26 | UAE National Day        | 2024 | 2024-12-01 | 2024-12-03 | 15 |
| 27 | Summer Sale             | 2024 | 2024-07-01 | 2024-08-31 | 10 |
| 28 | Ramadan Sale            | 2025 | 2025-03-01 | 2025-03-30 | 30 |
| 29 | Eid Al Fitr Sale        | 2025 | 2025-03-31 | 2025-04-02 | 30 |
| 30 | Eid Al Adha Sale        | 2025 | 2025-06-07 | 2025-06-09 | 25 |
| 31 | Dubai Shopping Festival | 2025 | 2025-01-01 | 2025-02-28 | 15 |
| 32 | Summer Sale             | 2025 | 2025-07-01 | 2025-08-31 | 10 |

|   | promo_type   | promo_id |
|---|--------------|----------|
| 0 | Ramadan      | 1        |
| 1 | Eid          | 2        |
| 2 | Eid          | 3        |
| 3 | DSF          | 4        |
| 4 | Black Friday | 5        |

```
print(promotions.columns.tolist())
print(promotions.head())
```

```
['promo_name', 'start_date', 'end_date', 'discount_pct', 'promo_type', 'promo_id']
promo_name start_date end_date discount_pct \
0 Ramadan Sale 2021 2021-04-13 2021-05-12 25
1 Eid Al Fitr Sale 2021 2021-05-13 2021-05-15 20
2 Eid Al Adha Sale 2021 2021-07-20 2021-07-22 25
3 Dubai Shopping Festival 2021 2021-01-01 2021-02-28 15
4 Black Friday 2021 2021-11-26 2021-11-28 30

promo_type promo_id
0 Ramadan 1
1 Eid 2
2 Eid 3
3 DSF 4
4 Black Friday 5
```

```
print("Number of promotions:", len(promotions))
print("Promotion date range:")
print("Start:", promotions["start_date"].min())
print("End:", promotions["end_date"].max())
```

Number of promotions: 33  
Promotion date range:  
Start: 2021-01-01 00:00:00  
End: 2025-08-31 00:00:00

```
model_data["month"] = model_data["week"].dt.month
model_data["quarter"] = model_data["week"].dt.quarter
model_data["year"] = model_data["week"].dt.year

print("Calendar features created successfully!")

print(
    model_data[
        ["week", "week_number", "month", "quarter", "year"]
    ].head(15)
)
```

Calendar features created successfully!

|    | week       | week_number | month | quarter | year |
|----|------------|-------------|-------|---------|------|
| 0  | 2020-12-28 | 53          | 12    | 4       | 2020 |
| 1  | 2021-01-04 | 1           | 1     | 1       | 2021 |
| 2  | 2021-01-11 | 2           | 1     | 1       | 2021 |
| 3  | 2021-01-18 | 3           | 1     | 1       | 2021 |
| 4  | 2021-01-25 | 4           | 1     | 1       | 2021 |
| 5  | 2021-02-01 | 5           | 2     | 1       | 2021 |
| 6  | 2021-02-08 | 6           | 2     | 1       | 2021 |
| 7  | 2021-02-15 | 7           | 2     | 1       | 2021 |
| 8  | 2021-02-22 | 8           | 2     | 1       | 2021 |
| 9  | 2021-03-01 | 9           | 3     | 1       | 2021 |
| 10 | 2021-03-08 | 10          | 3     | 1       | 2021 |
| 11 | 2021-03-15 | 11          | 3     | 1       | 2021 |
| 12 | 2021-03-22 | 12          | 3     | 1       | 2021 |
| 13 | 2021-03-29 | 13          | 3     | 1       | 2021 |
| 14 | 2021-04-05 | 14          | 4     | 2       | 2021 |

```
# Create weekly promotion features

promo_weeks = []

for _, promo in promotions.iterrows():
    weeks = pd.date_range(
        start=promo["start_date"].to_period("W-SUN").start_time
        end=promo["end_date"].to_period("W-SUN").start_time,
        freq="7D"
    )

    for week in weeks:
        promo_weeks.append({
            "week": week,
            "promo_type": promo["promo_type"],
            "discount_pct": promo["discount_pct"]
        })

promo_weekly = pd.DataFrame(promo_weeks)

print("Weekly promotion table created!")
```

```
print("Weekly promotion table created: ")
print("Shape:", promo_weekly.shape)
print(promo_weekly.head(10))
```

Weekly promotion table created!

Shape: (141, 3)

|   | week       | promo_type | discount_pct |
|---|------------|------------|--------------|
| 0 | 2021-04-12 | Ramadan    | 25           |
| 1 | 2021-04-19 | Ramadan    | 25           |
| 2 | 2021-04-26 | Ramadan    | 25           |
| 3 | 2021-05-03 | Ramadan    | 25           |
| 4 | 2021-05-10 | Ramadan    | 25           |
| 5 | 2021-05-10 | Eid        | 20           |
| 6 | 2021-07-19 | Eid        | 25           |
| 7 | 2020-12-28 | DSF        | 15           |
| 8 | 2021-01-04 | DSF        | 15           |
| 9 | 2021-01-11 | DSF        | 15           |

```
print("Unique promotion weeks:", promo_weekly["week"].nunique())
```

```
print("\nPromotion weeks by type:")
```

```
print(promo_weekly["promo_type"].value_counts())
```

```
print("\nSample:")
```

```
print(promo_weekly.head(15))
```

Unique promotion weeks: 132

Promotion weeks by type:

promo\_type

Summer Sale      48

DSF                47

Ramadan           25

Eid                12

National Day      5

Black Friday      4

Name: count, dtype: int64

Sample:

|    | week       | promo_type | discount_pct |
|----|------------|------------|--------------|
| 0  | 2021-04-12 | Ramadan    | 25           |
| 1  | 2021-04-19 | Ramadan    | 25           |
| 2  | 2021-04-26 | Ramadan    | 25           |
| 3  | 2021-05-03 | Ramadan    | 25           |
| 4  | 2021-05-10 | Ramadan    | 25           |
| 5  | 2021-05-10 | Eid        | 20           |
| 6  | 2021-07-19 | Eid        | 25           |
| 7  | 2020-12-28 | DSF        | 15           |
| 8  | 2021-01-04 | DSF        | 15           |
| 9  | 2021-01-11 | DSF        | 15           |
| 10 | 2021-01-18 | DSF        | 15           |
| 11 | 2021-01-25 | DSF        | 15           |
| 12 | 2021-02-01 | DSF        | 15           |
| 13 | 2021-02-08 | DSF        | 15           |
| 14 | 2021-02-15 | DSF        | 15           |

```
# Combine multiple promotions occurring in the same week
```

```

promo_weekly_agg = (
    promo_weekly
    .groupby("week", as_index=False)
    .agg(
        promo_flag=("promo_type", lambda x: 1),
        promo_discount=("discount_pct", "max"),
        promo_count=("promo_type", "count")
    )
)

print("Weekly promotion features aggregated successfully!")

print("Shape:", promo_weekly_agg.shape)

print(promo_weekly_agg.head(15))

```

Weekly promotion features aggregated successfully!

Shape: (132, 4)

|    | week       | promo_flag | promo_discount | promo_count |
|----|------------|------------|----------------|-------------|
| 0  | 2020-12-28 | 1          | 15             | 1           |
| 1  | 2021-01-04 | 1          | 15             | 1           |
| 2  | 2021-01-11 | 1          | 15             | 1           |
| 3  | 2021-01-18 | 1          | 15             | 1           |
| 4  | 2021-01-25 | 1          | 15             | 1           |
| 5  | 2021-02-01 | 1          | 15             | 1           |
| 6  | 2021-02-08 | 1          | 15             | 1           |
| 7  | 2021-02-15 | 1          | 15             | 1           |
| 8  | 2021-02-22 | 1          | 15             | 1           |
| 9  | 2021-04-12 | 1          | 25             | 1           |
| 10 | 2021-04-19 | 1          | 25             | 1           |
| 11 | 2021-04-26 | 1          | 25             | 1           |
| 12 | 2021-05-03 | 1          | 25             | 1           |
| 13 | 2021-05-10 | 1          | 25             | 2           |
| 14 | 2021-06-28 | 1          | 10             | 1           |

```

print("Unique weeks:", promo_weekly_agg["week"].nunique())
print("Rows:", len(promo_weekly_agg))

```

```

print("\nDuplicate weeks:")
print(
    promo_weekly_agg["week"]
    .duplicated()
    .sum()
)

```

Unique weeks: 132

Rows: 132

Duplicate weeks:

0

```

# Merge weekly promotion features into model data

```

```

model_data = model_data.merge(
    promo_weekly_agg,
    on="week",
    how="left"
)

```

```

)

print("Promotion features merged successfully!")
print("Model data shape:", model_data.shape)

print(
    model_data[
        [
            "week",
            "sku_id",
            "demand",
            "promo_flag",
            "promo_discount",
            "promo_count"
        ]
    ].head(20)
)

```

Promotion features merged successfully!

Model data shape: (50600, 28)

|    | week       | sku_id | demand | promo_flag | promo_discount | promo_count |
|----|------------|--------|--------|------------|----------------|-------------|
| 0  | 2020-12-28 | 1001   | 39     | 1.0        | 15.0           | 1.0         |
| 1  | 2021-01-04 | 1001   | 40     | 1.0        | 15.0           | 1.0         |
| 2  | 2021-01-11 | 1001   | 52     | 1.0        | 15.0           | 1.0         |
| 3  | 2021-01-18 | 1001   | 45     | 1.0        | 15.0           | 1.0         |
| 4  | 2021-01-25 | 1001   | 64     | 1.0        | 15.0           | 1.0         |
| 5  | 2021-02-01 | 1001   | 35     | 1.0        | 15.0           | 1.0         |
| 6  | 2021-02-08 | 1001   | 88     | 1.0        | 15.0           | 1.0         |
| 7  | 2021-02-15 | 1001   | 47     | 1.0        | 15.0           | 1.0         |
| 8  | 2021-02-22 | 1001   | 61     | 1.0        | 15.0           | 1.0         |
| 9  | 2021-03-01 | 1001   | 19     | NaN        | NaN            | NaN         |
| 10 | 2021-03-08 | 1001   | 20     | NaN        | NaN            | NaN         |
| 11 | 2021-03-15 | 1001   | 30     | NaN        | NaN            | NaN         |
| 12 | 2021-03-22 | 1001   | 24     | NaN        | NaN            | NaN         |
| 13 | 2021-03-29 | 1001   | 13     | NaN        | NaN            | NaN         |
| 14 | 2021-04-05 | 1001   | 36     | NaN        | NaN            | NaN         |
| 15 | 2021-04-12 | 1001   | 47     | 1.0        | 25.0           | 1.0         |
| 16 | 2021-04-19 | 1001   | 46     | 1.0        | 25.0           | 1.0         |
| 17 | 2021-04-26 | 1001   | 37     | 1.0        | 25.0           | 1.0         |
| 18 | 2021-05-03 | 1001   | 87     | 1.0        | 25.0           | 1.0         |
| 19 | 2021-05-10 | 1001   | 30     | 1.0        | 25.0           | 2.0         |

```

print("Missing promotion values:")
print(
    model_data[
        ["promo_flag", "promo_discount", "promo_count"]
    ].isnull().sum()
)

```

Missing promotion values:

```

promo_flag      24200
promo_discount   24200
promo_count      24200
dtype: int64

```

```
# Fill missing promotion values for non-promotion weeks
```

```

model_data["promo_flag"] = model_data["promo_flag"].fillna(0)
model_data["promo_discount"] = model_data["promo_discount"].fil
model_data["promo_count"] = model_data["promo_count"].fillna(0)

print("Missing promotion values handled successfully!")

print("\nMissing values after filling:")
print(
    model_data[
        ["promo_flag", "promo_discount", "promo_count"]
    ].isnull().sum()
)

```

Missing promotion values handled successfully!

Missing values after filling:

```

promo_flag      0
promo_discount  0
promo_count     0
dtype: int64

```

```

print("Final model data shape:", model_data.shape)

```

```

print("\nModel features:")
print(model_data.columns.tolist())

```

```

print("\nMissing values:")
print(model_data.isnull().sum())

```

Final model data shape: (50600, 28)

Model features:

```
['week', 'sku_id', 'demand', 'sales_value', 'avg_price', 'avg_discount', 'sku
```

Missing values:

```

week      0
sku_id    0
demand    0
sales_value  0
avg_price  0
avg_discount  0
sku_name  0
category  0
subcategory  0
brand     0
cost_price  0
week_number  0
lag_1     200
lag_2     400
lag_4     800
lag_8    1600
lag_13   2600
lag_26   5200
lag_52  10400
rolling_mean_4  800
rolling_mean_8  1600

```

```
rolling_mean_13    2600
month              0
quarter            0
year              0
promo_flag         0
promo_discount     0
promo_count        0
dtype: int64
```

```
# Remove rows without enough historical information

feature_columns = [
    "lag_1",
    "lag_2",
    "lag_4",
    "lag_8",
    "lag_13",
    "lag_26",
    "lag_52",
    "rolling_mean_4",
    "rolling_mean_8",
    "rolling_mean_13",
    "week_number",
    "month",
    "quarter",
    "year",
    "promo_flag",
    "promo_discount",
    "promo_count"
]

model_ready = model_data.dropna(
    subset=feature_columns
).copy()

print("Model-ready dataset shape:", model_ready.shape)

print("\nMissing values in features:")
print(model_ready[feature_columns].isnull().sum().sum())
```

```
Model-ready dataset shape: (40200, 28)
```

```
Missing values in features:
0
```

Start coding or [generate](#) with AI.

create train and test sets

```
# Time-based train-test split

train_data = model_ready[
    ~model_ready["week"].isin(test_weeks)
```

```

].copy()

test_data = model_ready[
    model_ready["week"].isin(test_weeks)
].copy()

print("Training data shape:", train_data.shape)
print("Test data shape:", test_data.shape)

print("\nTraining period:")
print("Start:", train_data["week"].min())
print("End:", train_data["week"].max())

print("\nTest period:")
print("Start:", test_data["week"].min())
print("End:", test_data["week"].max())

```

```

Training data shape: (39000, 28)
Test data shape: (1200, 28)

```

```

Training period:
Start: 2021-12-27 00:00:00
End: 2025-09-15 00:00:00

```

```

Test period:
Start: 2025-09-22 00:00:00
End: 2025-10-27 00:00:00

```

```

# Define features and target

```

```

feature_columns = [
    "lag_1",
    "lag_2",
    "lag_4",
    "lag_8",
    "lag_13",
    "lag_26",
    "lag_52",
    "rolling_mean_4",
    "rolling_mean_8",
    "rolling_mean_13",
    "week_number",
    "month",
    "quarter",
    "year",
    "promo_flag",
    "promo_discount",
    "promo_count"
]

```

```

X_train = train_data[feature_columns]
y_train = train_data["demand"]

```

```

X_test = test_data[feature_columns]
y_test = test_data["demand"]

```

```
print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)
print("X_test shape:", X_test.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (39000, 17)
y_train shape: (39000,)
X_test shape: (1200, 17)
y_test shape: (1200,)
```

```
from sklearn.ensemble import GradientBoostingRegressor

# Create Gradient Boosting model
gb_model = GradientBoostingRegressor(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=3,
    random_state=42
)

# Train the model
gb_model.fit(X_train, y_train)

print("Gradient Boosting model trained successfully!")
```

```
Gradient Boosting model trained successfully!
```

## make predictions

```
# Generate predictions on the test data

y_pred = gb_model.predict(X_test)

print("Predictions generated successfully!")
print("Number of predictions:", len(y_pred))

print("\nFirst 10 predictions:")
print(y_pred[:10])
```

```
Predictions generated successfully!
Number of predictions: 1200

First 10 predictions:
[19.60138235 19.29102586 19.49316647 19.45880313 19.51985611 19.67570964
 18.64173602 19.59640761 19.92677559 19.68234547]
```

```
# Calculate WAPE and Bias for Gradient Boosting

actual = y_test
forecast = y_pred

wape_gb = (
```

```

    np.sum(np.abs(actual - forecast))
    / np.sum(actual)
) * 100

bias_gb = (
    np.sum(forecast - actual)
    / np.sum(actual)
) * 100

print("=" * 50)
print("GRADIENT BOOSTING RESULTS")
print("=" * 50)
print(f"WAPE: {wape_gb:.2f}%")
print(f"Bias: {bias_gb:.2f}%")

print("\nBaseline Comparison")
print("-" * 50)
print("Seasonal Naive WAPE: 45.56%")
print(f"Gradient Boosting WAPE: {wape_gb:.2f}%")

```

```

=====
GRADIENT BOOSTING RESULTS
=====
WAPE: 32.99%
Bias: 9.56%

Baseline Comparison
-----
Seasonal Naive WAPE: 45.56%
Gradient Boosting WAPE: 32.99%

```

```

# Rolling-origin validation setup

validation_weeks = [
    "2025-06-16",
    "2025-07-14",
    "2025-08-11"
]

print("Validation origins:")
for week in validation_weeks:
    print(week)

```

```

Validation origins:
2025-06-16
2025-07-14
2025-08-11

```

```

# Rolling-origin validation

from sklearn.ensemble import GradientBoostingRegressor

validation_results = []

```

```

for origin in validation_weeks:

    origin = pd.to_datetime(origin)

    # Training data: everything before the validation origin
    train_cv = model_ready[
        model_ready["week"] < origin
    ].copy()

    # Validation data: next 6 weeks
    validation_cv = model_ready[
        (model_ready["week"] >= origin) &
        (model_ready["week"] < origin + pd.Timedelta(weeks=6))
    ].copy()

    # Features and target
    X_train_cv = train_cv[feature_columns]
    y_train_cv = train_cv["demand"]

    X_val_cv = validation_cv[feature_columns]
    y_val_cv = validation_cv["demand"]

    # Train model
    cv_model = GradientBoostingRegressor(
        n_estimators=200,
        learning_rate=0.05,
        max_depth=3,
        random_state=42
    )

    cv_model.fit(X_train_cv, y_train_cv)

    # Predict
    y_pred_cv = cv_model.predict(X_val_cv)

    # Metrics
    wape_cv = (
        np.sum(np.abs(y_val_cv - y_pred_cv))
        / np.sum(y_val_cv)
    ) * 100

    bias_cv = (
        np.sum(y_pred_cv - y_val_cv)
        / np.sum(y_val_cv)
    ) * 100

    validation_results.append({
        "origin": origin,
        "WAPE": wape_cv,
        "Bias": bias_cv
    })

validation_results = pd.DataFrame(validation_results)

```

```
print("=" * 50)
print("ROLLING-ORIGIN VALIDATION RESULTS")
print("=" * 50)
print(validation_results)
```

```
=====
ROLLING-ORIGIN VALIDATION RESULTS
=====
```

|   | origin     | WAPE      | Bias      |
|---|------------|-----------|-----------|
| 0 | 2025-06-16 | 25.857502 | 0.757751  |
| 1 | 2025-07-14 | 26.418844 | -0.675923 |
| 2 | 2025-08-11 | 28.756495 | 0.694803  |

```
# Calculate average rolling-validation performance
```

```
avg_wape = validation_results["WAPE"].mean()
avg_bias = validation_results["Bias"].mean()
```

```
print("=" * 50)
print("AVERAGE ROLLING-ORIGIN VALIDATION")
print("=" * 50)
print(f"Average WAPE: {avg_wape:.2f}%")
print(f"Average Bias: {avg_bias:.2f}%")
```

```
print("\nComparison with Seasonal Naive:")
print(f"Seasonal Naive WAPE: 45.56%")
print(f"Gradient Boosting Average WAPE: {avg_wape:.2f}%")
```

```
=====
AVERAGE ROLLING-ORIGIN VALIDATION
=====
```

Average WAPE: 27.01%  
Average Bias: 0.26%

```
Comparison with Seasonal Naive:
Seasonal Naive WAPE: 45.56%
Gradient Boosting Average WAPE: 27.01%
```

```
# Check the latest historical data available

print("Latest historical week:", model_data["week"].max())
print("Number of SKUs:", model_data["sku_id"].nunique())
print("Historical rows:", len(model_data))

print("\nLatest 10 weeks:")
print(
    model_data["week"]
    .drop_duplicates()
    .sort_values()
    .tail(10)
)
```

```
Latest historical week: 2025-10-27 00:00:00
Number of SKUs: 200
```

Historical rows: 50600

Latest 10 weeks:

```
243    2025-08-25
244    2025-09-01
245    2025-09-08
246    2025-09-15
247    2025-09-22
248    2025-09-29
249    2025-10-06
250    2025-10-13
251    2025-10-20
252    2025-10-27
```

Name: week, dtype: datetime64[ns]

```
# Create the next 6 forecast weeks
```

```
last_week = model_data["week"].max()
```

```
future_weeks = pd.date_range(
    start=last_week + pd.Timedelta(weeks=1),
    periods=6,
    freq="7D"
)
```

```
print("Future forecast weeks:")
print(future_weeks)
```

Future forecast weeks:

```
DatetimeIndex(['2025-11-03', '2025-11-10', '2025-11-17', '2025-11-24',
               '2025-12-01', '2025-12-08'],
              dtype='datetime64[ns]', freq='7D')
```

```
# Create future SKU × week combinations
```

```
future_grid = pd.MultiIndex.from_product(
    [future_weeks, model_data["sku_id"].unique()],
    names=["week", "sku_id"]
).to_frame(index=False)
```

```
print("Future SKU-week grid created!")
print("Shape:", future_grid.shape)
```

```
print("\nNumber of future weeks:", future_grid["week"].nunique())
print("Number of SKUs:", future_grid["sku_id"].nunique())
```

```
print("\nFirst 10 rows:")
print(future_grid.head(10))
```

Future SKU-week grid created!  
Shape: (1200, 2)

Number of future weeks: 6  
Number of SKUs: 200

First 10 rows:

|   | week       | sku_id |
|---|------------|--------|
| 0 | 2025-11-03 | 1001   |
| 1 | 2025-11-03 | 1002   |
| 2 | 2025-11-03 | 1003   |
| 3 | 2025-11-03 | 1004   |
| 4 | 2025-11-03 | 1005   |
| 5 | 2025-11-03 | 1006   |
| 6 | 2025-11-03 | 1007   |
| 7 | 2025-11-03 | 1008   |
| 8 | 2025-11-03 | 1009   |
| 9 | 2025-11-03 | 1010   |

```
# Add calendar features for future weeks

future_grid["week_number"] = (
    future_grid["week"].dt.isocalendar().week.astype(int)
)

future_grid["month"] = future_grid["week"].dt.month
future_grid["quarter"] = future_grid["week"].dt.quarter
future_grid["year"] = future_grid["week"].dt.year

print("Calendar features added!")

print("\nSample:")
print(
    future_grid[
        ["week", "sku_id", "week_number", "month", "quarter", "year"]
    ].head(10)
)
```

Calendar features added!

Sample:

|   | week       | sku_id | week_number | month | quarter | year |
|---|------------|--------|-------------|-------|---------|------|
| 0 | 2025-11-03 | 1001   | 45          | 11    | 4       | 2025 |
| 1 | 2025-11-03 | 1002   | 45          | 11    | 4       | 2025 |
| 2 | 2025-11-03 | 1003   | 45          | 11    | 4       | 2025 |
| 3 | 2025-11-03 | 1004   | 45          | 11    | 4       | 2025 |
| 4 | 2025-11-03 | 1005   | 45          | 11    | 4       | 2025 |
| 5 | 2025-11-03 | 1006   | 45          | 11    | 4       | 2025 |
| 6 | 2025-11-03 | 1007   | 45          | 11    | 4       | 2025 |
| 7 | 2025-11-03 | 1008   | 45          | 11    | 4       | 2025 |
| 8 | 2025-11-03 | 1009   | 45          | 11    | 4       | 2025 |
| 9 | 2025-11-03 | 1010   | 45          | 11    | 4       | 2025 |

```
# Add promotion features for future weeks

future_grid["promo_flag"] = 0
future_grid["promo_discount"] = 0
future_grid["promo_count"] = 0

print("Future promotion features added!")

print("\nPromotion feature summary:")
print(
```

```

future_grid[
  ["promo_flag", "promo_discount", "promo_count"]
].describe()
)

print("\nUnique values:")
print("promo_flag:", future_grid["promo_flag"].unique())
print("promo_discount:", future_grid["promo_discount"].unique())
print("promo_count:", future_grid["promo_count"].unique())

```

Future promotion features added!

Promotion feature summary:

|       | promo_flag | promo_discount | promo_count |
|-------|------------|----------------|-------------|
| count | 1200.0     | 1200.0         | 1200.0      |
| mean  | 0.0        | 0.0            | 0.0         |
| std   | 0.0        | 0.0            | 0.0         |
| min   | 0.0        | 0.0            | 0.0         |
| 25%   | 0.0        | 0.0            | 0.0         |
| 50%   | 0.0        | 0.0            | 0.0         |
| 75%   | 0.0        | 0.0            | 0.0         |
| max   | 0.0        | 0.0            | 0.0         |

Unique values:

promo\_flag: [0]

promo\_discount: [0]

promo\_count: [0]

```

from sklearn.ensemble import GradientBoostingRegressor

# Train final model using all available historical data

X_full = model_ready[feature_columns]
y_full = model_ready["demand"]

final_model = GradientBoostingRegressor(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=3,
    random_state=42
)

final_model.fit(X_full, y_full)

print("Final model trained successfully!")
print("Training rows:", X_full.shape[0])
print("Number of features:", X_full.shape[1])

```

Final model trained successfully!

Training rows: 40200

Number of features: 17

# Prepare historical demand for recursive forecasting

```
history = model_data[["week", "sku_id", "demand"]].copy()
```

```

history = history.sort_values(
    ["sku_id", "week"]
).reset_index(drop=True)

print("Historical demand prepared!")
print("Shape:", history.shape)
print("Date range:", history["week"].min(), "to", history["week"].max())
print("Number of SKUs:", history["sku_id"].nunique())

print("\nLast 5 historical records:")
print(history.tail())

```

```

Historical demand prepared!
Shape: (50600, 3)
Date range: 2020-12-28 00:00:00 to 2025-10-27 00:00:00
Number of SKUs: 200

```

```

Last 5 historical records:

```

|       | week       | sku_id | demand |
|-------|------------|--------|--------|
| 50595 | 2025-09-29 | 1200   | 7      |
| 50596 | 2025-10-06 | 1200   | 20     |
| 50597 | 2025-10-13 | 1200   | 15     |
| 50598 | 2025-10-20 | 1200   | 19     |
| 50599 | 2025-10-27 | 1200   | 26     |

```

# Recursive 6-week forecasting

forecast_results = []

# Copy historical data so we can append predictions
history_recursive = history.copy()

for future_week in future_weeks:

    future_rows = []

    for sku in history_recursive["sku_id"].unique():

        sku_history = history_recursive[
            history_recursive["sku_id"] == sku
        ].sort_values("week")

        demand_values = sku_history["demand"].values

        # Create lag features
        lag_1 = demand_values[-1]
        lag_2 = demand_values[-2]
        lag_4 = demand_values[-4]
        lag_8 = demand_values[-8]
        lag_13 = demand_values[-13]
        lag_26 = demand_values[-26]
        lag_52 = demand_values[-52]

        # Create rolling mean features
        rolling_mean_4 = np.mean(demand_values[-4:])

```

```

rolling_mean_8 = np.mean(demand_values[-8:])
rolling_mean_13 = np.mean(demand_values[-13:])

# Calendar features
week_number = future_week.isocalendar().week
month = future_week.month
quarter = future_week.quarter
year = future_week.year

# Promotion features
promo_flag = 0
promo_discount = 0
promo_count = 0

# Create model input
X_future = pd.DataFrame([
    "lag_1": lag_1,
    "lag_2": lag_2,
    "lag_4": lag_4,
    "lag_8": lag_8,
    "lag_13": lag_13,
    "lag_26": lag_26,
    "lag_52": lag_52,
    "rolling_mean_4": rolling_mean_4,
    "rolling_mean_8": rolling_mean_8,
    "rolling_mean_13": rolling_mean_13,
    "week_number": week_number,
    "month": month,
    "quarter": quarter,
    "year": year,
    "promo_flag": promo_flag,
    "promo_discount": promo_discount,
    "promo_count": promo_count
])

# Make prediction
prediction = final_model.predict(X_future)[0]

# Demand cannot be negative
prediction = max(0, prediction)

future_rows.append({
    "week": future_week,
    "sku_id": sku,
    "forecast_demand": prediction
})

# Store forecasts
week_forecasts = pd.DataFrame(future_rows)
forecast_results.append(week_forecasts)

# Add predictions to history for the next iteration
predicted_history = week_forecasts.rename(
    columns={"forecast_demand": "demand"}
)

```

```

        history_recursive = pd.concat(
            [history_recursive, predicted_history],
            ignore_index=True
        )

# Combine all 6 weeks
future_forecast = pd.concat(
    forecast_results,
    ignore_index=True
)

print("Recursive forecasting completed!")
print("Forecast shape:", future_forecast.shape)

print("\nForecast weeks:")
print(future_forecast["week"].unique())

print("\nFirst 10 forecasts:")
print(future_forecast.head(10))

```

Recursive forecasting completed!  
Forecast shape: (1200, 3)

Forecast weeks:  
<DatetimeArray>  
['2025-11-03 00:00:00', '2025-11-10 00:00:00', '2025-11-17 00:00:00',  
 '2025-11-24 00:00:00', '2025-12-01 00:00:00', '2025-12-08 00:00:00']  
Length: 6, dtype: datetime64[ns]

First 10 forecasts:

|   | week       | sku_id | forecast_demand |
|---|------------|--------|-----------------|
| 0 | 2025-11-03 | 1001   | 18.667519       |
| 1 | 2025-11-03 | 1002   | 18.085064       |
| 2 | 2025-11-03 | 1003   | 17.815627       |
| 3 | 2025-11-03 | 1004   | 18.231688       |
| 4 | 2025-11-03 | 1005   | 18.775419       |
| 5 | 2025-11-03 | 1006   | 18.018202       |
| 6 | 2025-11-03 | 1007   | 18.317055       |
| 7 | 2025-11-03 | 1008   | 17.897878       |
| 8 | 2025-11-03 | 1009   | 18.015264       |
| 9 | 2025-11-03 | 1010   | 17.938547       |

```

# Add SKU information to the future forecast

future_forecast = future_forecast.merge(
    skus[
        [
            "sku_id",
            "sku_name",
            "category",
            "subcategory",
            "brand",
            "cost_price"
        ]
    ],
    on="sku_id",
    how="left",
)

```

```

on= sku_id ,
how="left"
)

print("SKU information added!")

print("Shape:", future_forecast.shape)

print("\nMissing values:")
print(future_forecast.isna().sum())

print("\nFirst 10 rows:")
print(future_forecast.head(10))

```

SKU information added!  
Shape: (1200, 8)

Missing values:

```

week          0
sku_id        0
forecast_demand  0
sku_name      0
category      0
subcategory   0
brand         0
cost_price    0
dtype: int64

```

First 10 rows:

|   | week       | sku_id | forecast_demand | sku_name \                       |
|---|------------|--------|-----------------|----------------------------------|
| 0 | 2025-11-03 | 1001   | 18.667519       | Electronics_Chargers_1001        |
| 1 | 2025-11-03 | 1002   | 18.085064       | Dairy_Cream_1002                 |
| 2 | 2025-11-03 | 1003   | 17.815627       | Household_Cleaning_Supplies_1003 |
| 3 | 2025-11-03 | 1004   | 18.231688       | Beverages_Juices_1004            |
| 4 | 2025-11-03 | 1005   | 18.775419       | Household_Paper_Products_1005    |
| 5 | 2025-11-03 | 1006   | 18.018202       | Beverages_Coffee_1006            |
| 6 | 2025-11-03 | 1007   | 18.317055       | Personal_Care_Shampoo_1007       |
| 7 | 2025-11-03 | 1008   | 17.897878       | Personal_Care_Shampoo_1008       |
| 8 | 2025-11-03 | 1009   | 18.015264       | Electronics_Batteries_1009       |
| 9 | 2025-11-03 | 1010   | 17.938547       | Grocery_Rice_1010                |

|   | category      | subcategory       | brand         | cost_price |
|---|---------------|-------------------|---------------|------------|
| 0 | Electronics   | Chargers          | Budget        | 103.44     |
| 1 | Dairy         | Cream             | Local         | 8.96       |
| 2 | Household     | Cleaning Supplies | BlueMart      | 7.47       |
| 3 | Beverages     | Juices            | International | 17.58      |
| 4 | Household     | Paper Products    | Budget        | 13.68      |
| 5 | Beverages     | Coffee            | Premium       | 9.80       |
| 6 | Personal Care | Shampoo           | BlueMart      | 51.47      |
| 7 | Personal Care | Shampoo           | Premium       | 31.77      |
| 8 | Electronics   | Batteries         | Local         | 61.82      |
| 9 | Grocery       | Rice              | Budget        | 10.05      |

# Forecast summary by week

```

weekly_forecast_summary = (
    future_forecast
    .groupby("week", as_index=False)

```

```

        .agg(
            total_forecast_demand=("forecast_demand", "sum"),
            average_forecast_demand=("forecast_demand", "mean"),
            minimum_forecast_demand=("forecast_demand", "min"),
            maximum_forecast_demand=("forecast_demand", "max")
        )
    )

print("Weekly Forecast Summary:")
print(weekly_forecast_summary.round(2))

```

Weekly Forecast Summary:

|   | week       | total_forecast_demand | average_forecast_demand \ |
|---|------------|-----------------------|---------------------------|
| 0 | 2025-11-03 | 3642.00               | 18.21                     |
| 1 | 2025-11-10 | 3671.94               | 18.36                     |
| 2 | 2025-11-17 | 3665.95               | 18.33                     |
| 3 | 2025-11-24 | 3694.83               | 18.47                     |
| 4 | 2025-12-01 | 4558.87               | 22.79                     |
| 5 | 2025-12-08 | 4545.90               | 22.73                     |

|   | minimum_forecast_demand | maximum_forecast_demand |
|---|-------------------------|-------------------------|
| 0 | 17.72                   | 19.21                   |
| 1 | 18.06                   | 18.88                   |
| 2 | 17.57                   | 18.72                   |
| 3 | 17.54                   | 20.38                   |
| 4 | 20.72                   | 28.23                   |
| 5 | 20.77                   | 26.01                   |

```

# Top 10 SKUs by total forecast demand

top_forecast_skus = (
    future_forecast
    .groupby(
        ["sku_id", "sku_name", "category"],
        as_index=False
    )
    .agg(
        total_forecast_demand=("forecast_demand", "sum"),
        average_weekly_demand=("forecast_demand", "mean")
    )
    .sort_values(
        "total_forecast_demand",
        ascending=False
    )
    .head(10)
)

print("Top 10 Forecasted SKUs:")
print(top_forecast_skus.round(2))

```

Top 10 Forecasted SKUs:

|     | sku_id | sku_name                         | category \ |
|-----|--------|----------------------------------|------------|
| 127 | 1128   | Grocery_Spices_1128              | Grocery    |
| 197 | 1198   | Dairy_Cheese_1198                | Dairy      |
| 21  | 1022   | Household_Cleaning_Supplies_1022 | Household  |

|     |      |                                     |               |
|-----|------|-------------------------------------|---------------|
| 158 | 1159 | Electronics_Batteries_1159          | Electronics   |
| 154 | 1155 | Electronics_Mobile Accessories_1155 | Electronics   |
| 186 | 1187 | Personal_Care_Skincare_1187         | Personal Care |
| 117 | 1118 | Dairy_Milk_1118                     | Dairy         |
| 51  | 1052 | Snacks_Crackers_1052                | Snacks        |
| 54  | 1055 | Dairy_Butter_1055                   | Dairy         |
| 85  | 1086 | Grocery_Rice_1086                   | Grocery       |

|     | total_forecast_demand | average_weekly_demand |
|-----|-----------------------|-----------------------|
| 127 | 124.18                | 20.70                 |
| 197 | 124.17                | 20.69                 |
| 21  | 123.72                | 20.62                 |
| 158 | 122.42                | 20.40                 |
| 154 | 122.32                | 20.39                 |
| 186 | 121.47                | 20.25                 |
| 117 | 121.32                | 20.22                 |
| 51  | 120.54                | 20.09                 |
| 54  | 120.54                | 20.09                 |
| 85  | 120.19                | 20.03                 |

```
# Bottom 10 SKUs by total forecast demand

bottom_forecast_skus = (
    future_forecast
    .groupby(
        ["sku_id", "sku_name", "category"],
        as_index=False
    )
    .agg(
        total_forecast_demand=("forecast_demand", "sum"),
        average_weekly_demand=("forecast_demand", "mean")
    )
    .sort_values(
        "total_forecast_demand",
        ascending=True
    )
    .head(10)
)

print("Bottom 10 Forecasted SKUs:")
print(bottom_forecast_skus.round(2))
```

Bottom 10 Forecasted SKUs:

|     | sku_id | sku_name                      | category \    |
|-----|--------|-------------------------------|---------------|
| 57  | 1058   | Personal_Care_Soap_1058       | Personal Care |
| 152 | 1153   | Electronics_Batteries_1153    | Electronics   |
| 8   | 1009   | Electronics_Batteries_1009    | Electronics   |
| 37  | 1038   | Electronics_Cables_1038       | Electronics   |
| 120 | 1121   | Grocery_Rice_1121             | Grocery       |
| 110 | 1111   | Snacks_Nuts_1111              | Snacks        |
| 121 | 1122   | Dairy_Yogurt_1122             | Dairy         |
| 159 | 1160   | Snacks_Nuts_1160              | Snacks        |
| 171 | 1172   | Electronics_Chargers_1172     | Electronics   |
| 98  | 1099   | Personal_Care_Toothpaste_1099 | Personal Care |

|    | total_forecast_demand | average_weekly_demand |
|----|-----------------------|-----------------------|
| 57 | 115.69                | 19.28                 |

|     |        |       |
|-----|--------|-------|
| 152 | 115.72 | 19.29 |
| 8   | 115.75 | 19.29 |
| 37  | 115.79 | 19.30 |
| 120 | 116.51 | 19.42 |
| 110 | 116.87 | 19.48 |
| 121 | 117.01 | 19.50 |
| 159 | 117.01 | 19.50 |
| 171 | 117.06 | 19.51 |
| 98  | 117.08 | 19.51 |

Start coding or generate with AI.

Start coding or generate with AI.